RICE SOCIAL SCIENCES
Department of Linguistics

large language models

LING 430 Computational Linguistics Fall 2026
lecture 12b
Nov 12 2026

Siyu Liang
siyu.liang@rice.edu

logistics

- **HW 6 due tonight, 11:59 pm**

last time

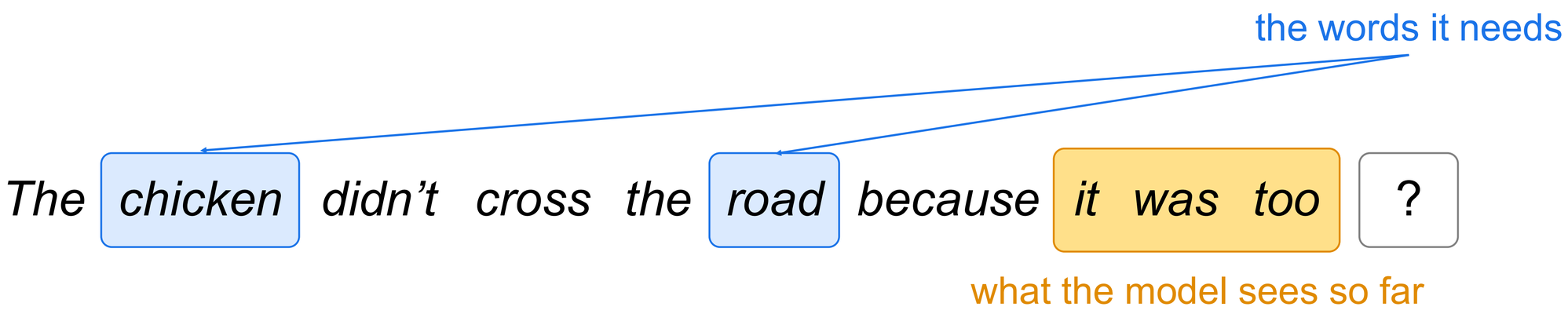
- neural networks
  - neural units
  - the XOR problem
  - feedforward networks
  - classification
  - neural language models

agenda

- attention
- the transformer block
- decoding
- pretraining
- instruction tuning

recap

- last time we left with a language model based on a feedforward network
- but it had a short memory: it only saw the last three words
- what comes next here?



large language models

road map

- to work towards a large language model, we need four things
  - **attention**, so each word can look at all the words before it
  - the **transformer**, a network built around attention
  - **pretraining** on a huge amount of text
  - **instruction tuning**, which turns a language model into an assistant helpful for us

large language models

- attention and the transformer come from one paper (Vaswani et al. 2017)

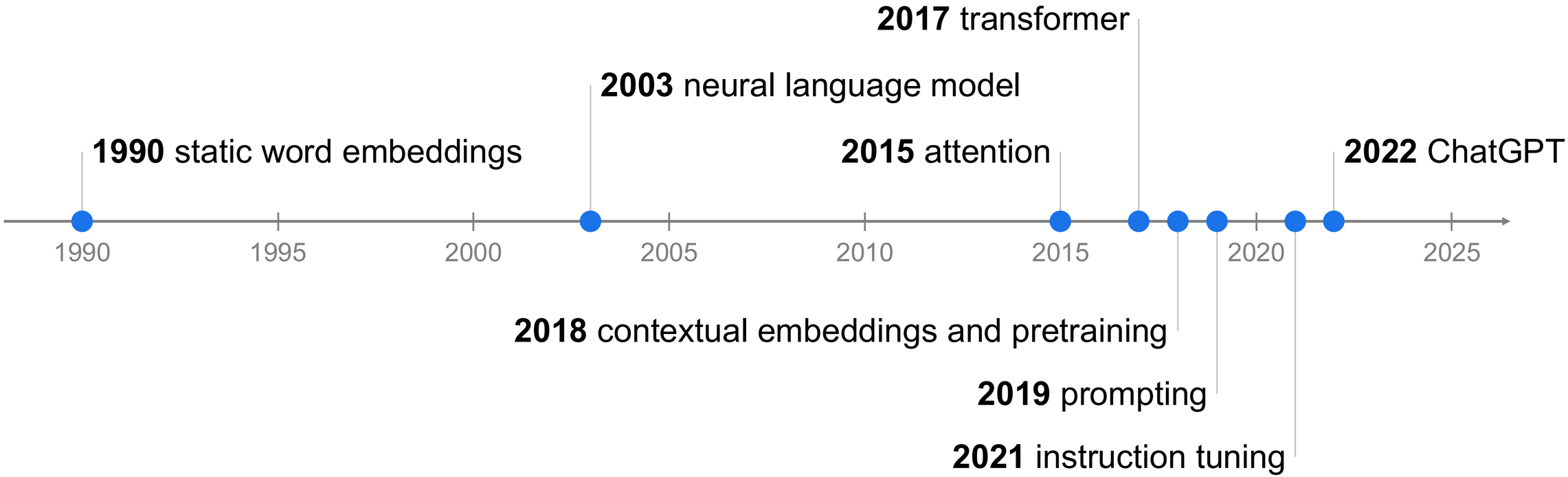
Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com



large language models

a very approximate timeline

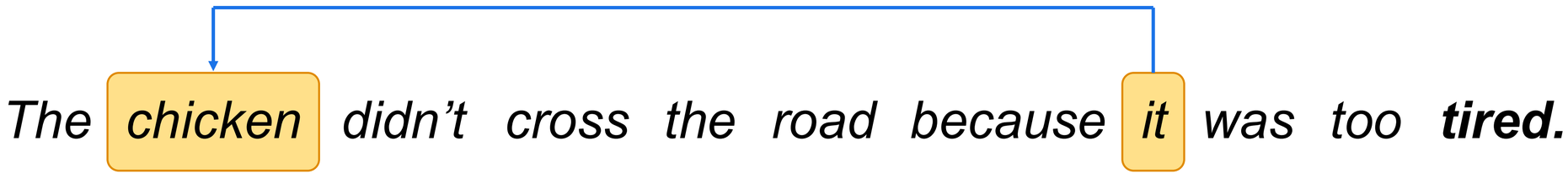


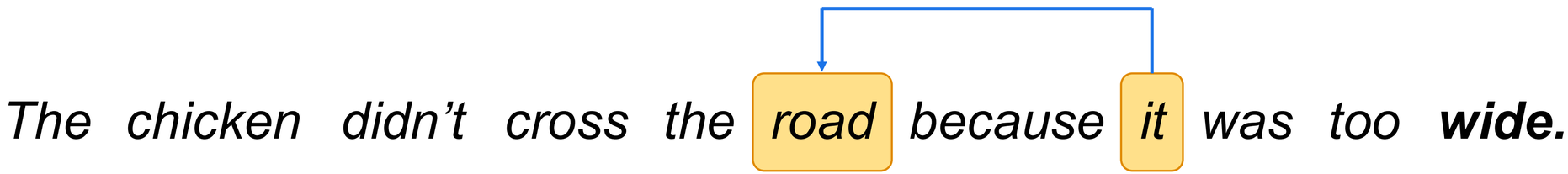
attention

attention

static embeddings

- what does *it* refer to?





attention

static embeddings

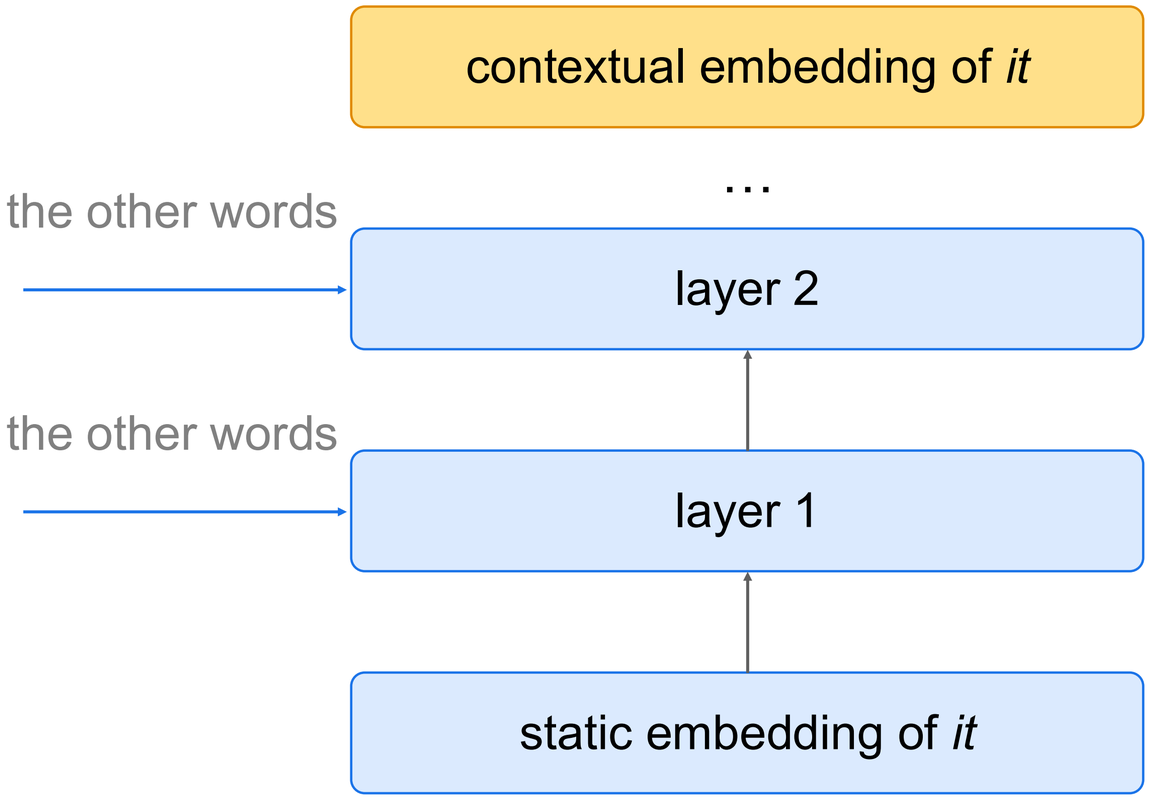
- so far, each word has had one fixed vector, a **static embedding**
  - so *it* has the same vector in both sentences
- we want the vector for *it* to depend on the words around it

sentence	**vector for *it***
… because it was too tired	[0.2, −0.4, 0.7, …]
… because it was too wide	[0.2, −0.4, 0.7, …]

attention

contextual embeddings

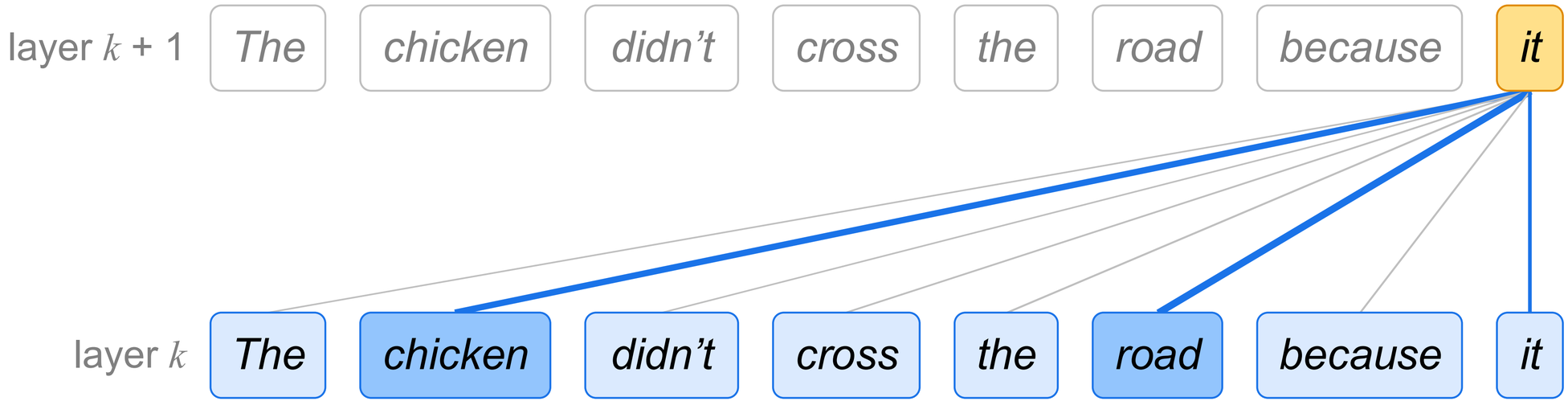
- a **contextual embedding** is a vector for a word in its context
  - a different context gives a different vector
- a transformer builds it layer by layer



attention

intuition

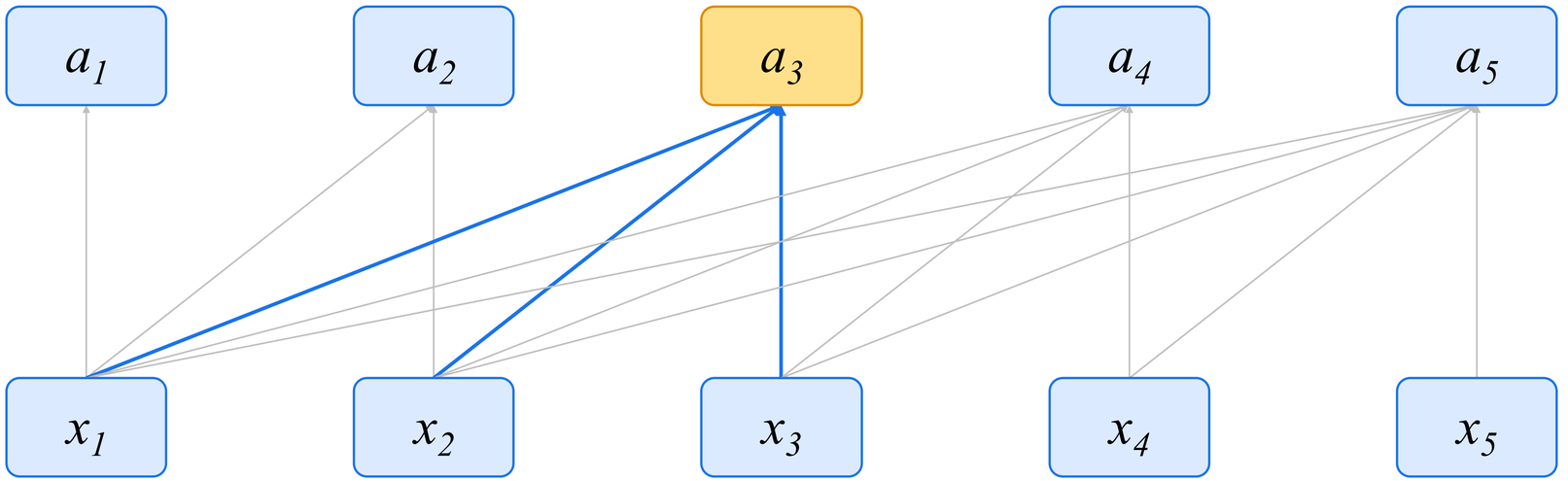
- attention builds the new vector for a word by mixing in the vectors of the words before it
- some words matter more than others
  - we say *it* **attends to** *chicken* and *road*



attention

direction

- a language model predicts the next word, so a word can only attend to the words before it
  - guessing the next word is pretty simple if you already know it!



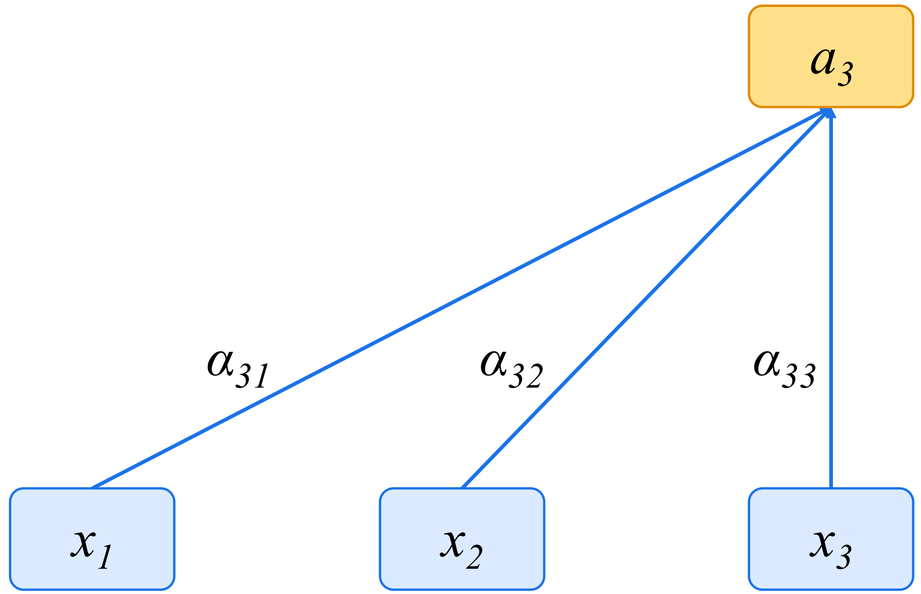
attention

weighted sum

- so what is attention exactly?
  - a weighted sum of the vectors so far

$$a_i = \sum_{j \leq i} \alpha_{\mathrm{ij}} \cdot x_j$$

- the weights α say how much each word matters

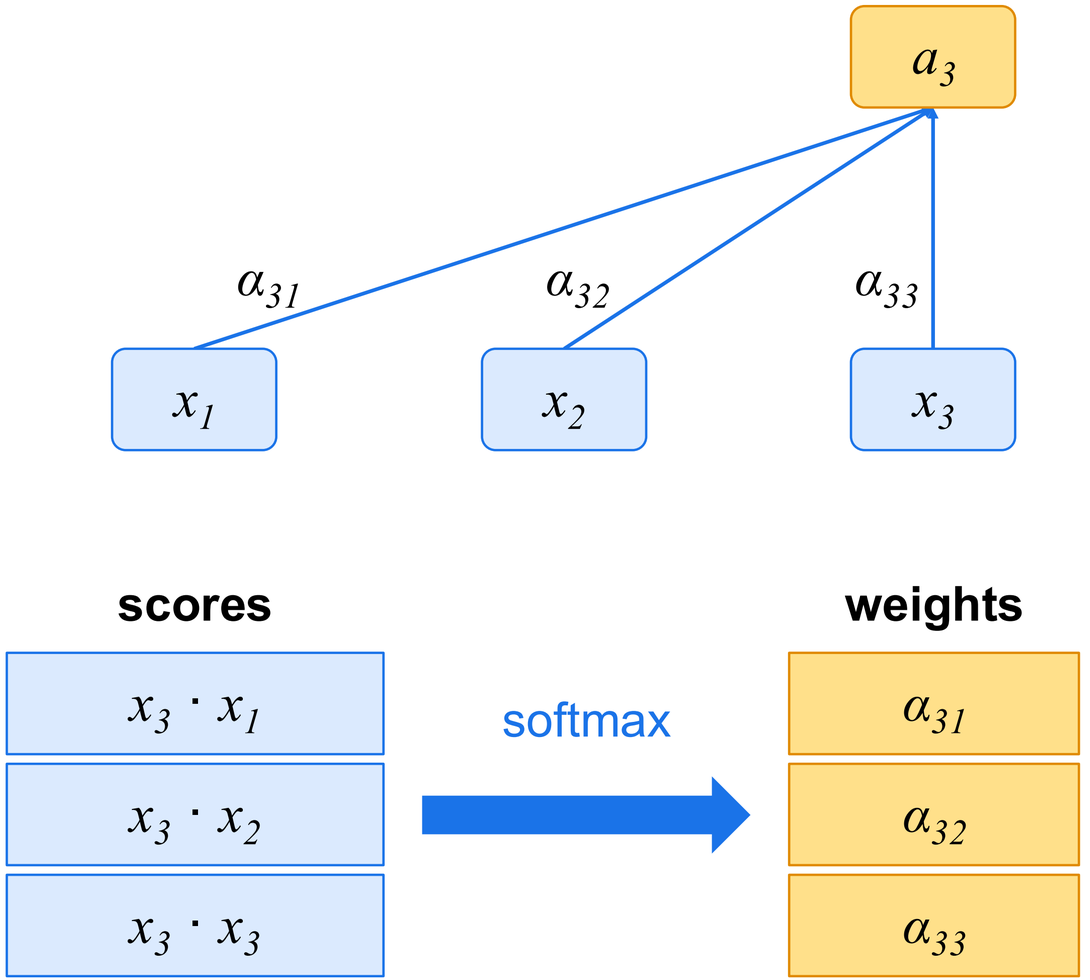


attention

weights

- where do the weights come from?
- we score each earlier word by its dot product with the current word
- then the softmax turns the scores into weights

$$\alpha_{\mathrm{ij}} = \mathrm{softmax}\left(x_i \cdot x_j\right)$$



attention

query, key & value

- in the weighted sum for attnetion, a word plays three different roles
- compared to a search

role	when you search for a video	in attention
query	what you type in the search box	what the current word is looking for
key	the title of each video	how each word describes itself
value	the video you end up watching	what each word passes on

attention

query, key & value

- **query**: what the current word is looking for

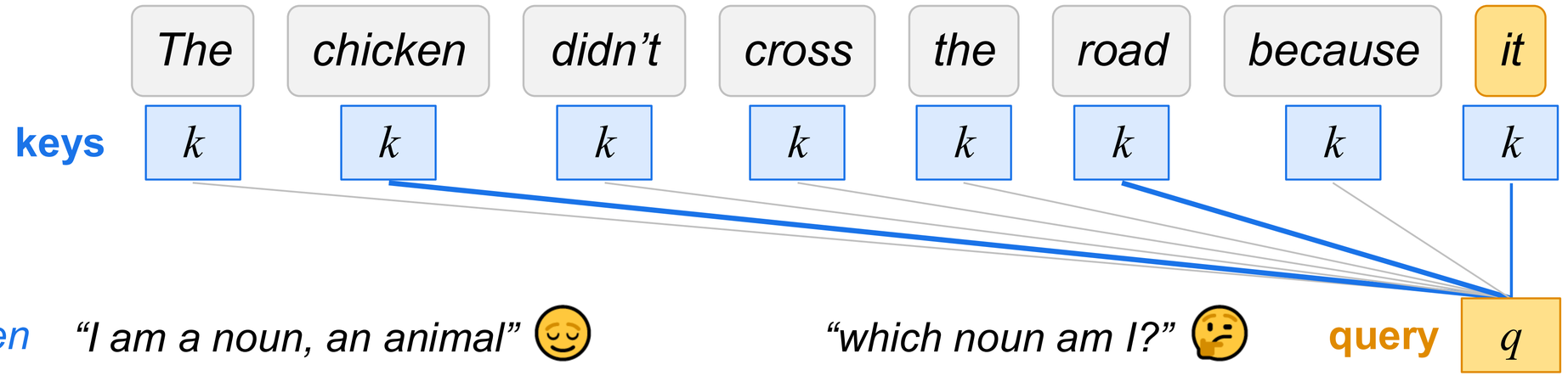
The *chicken* *didn't* *cross* *the* *road* *because* *it*

"which noun am I?" 🤔 **query** q

attention

query, key & value

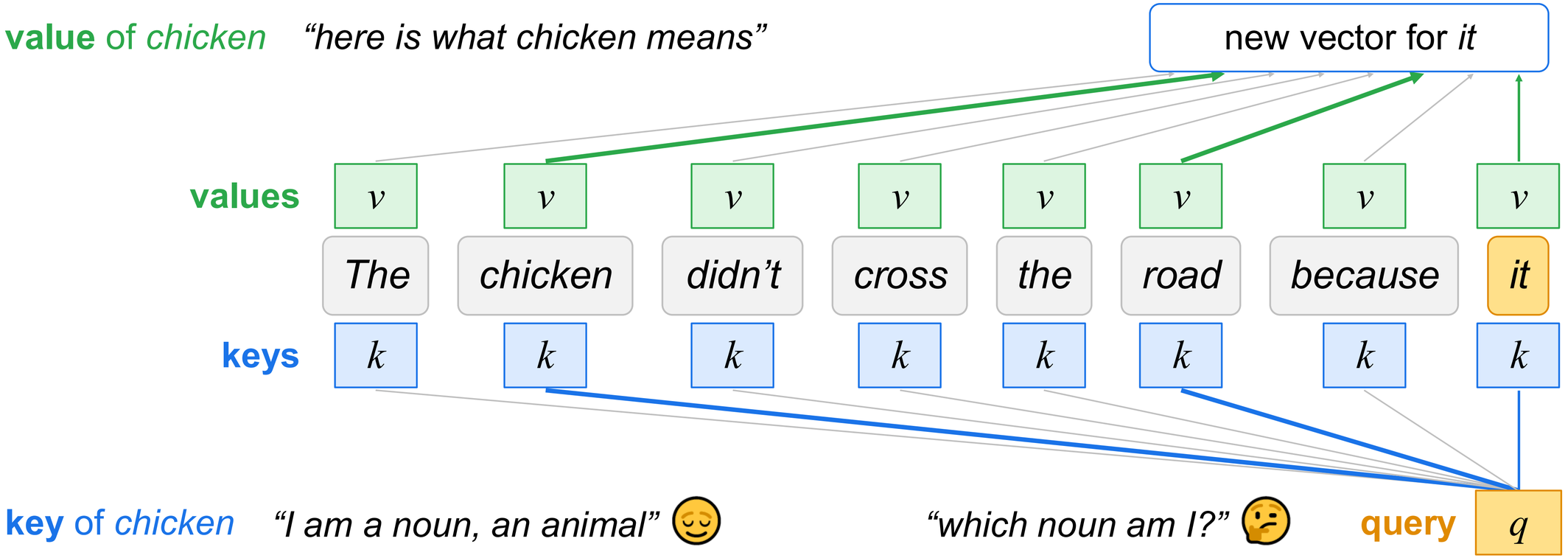
- **key**: how each word describes itself



attention

query, key & value

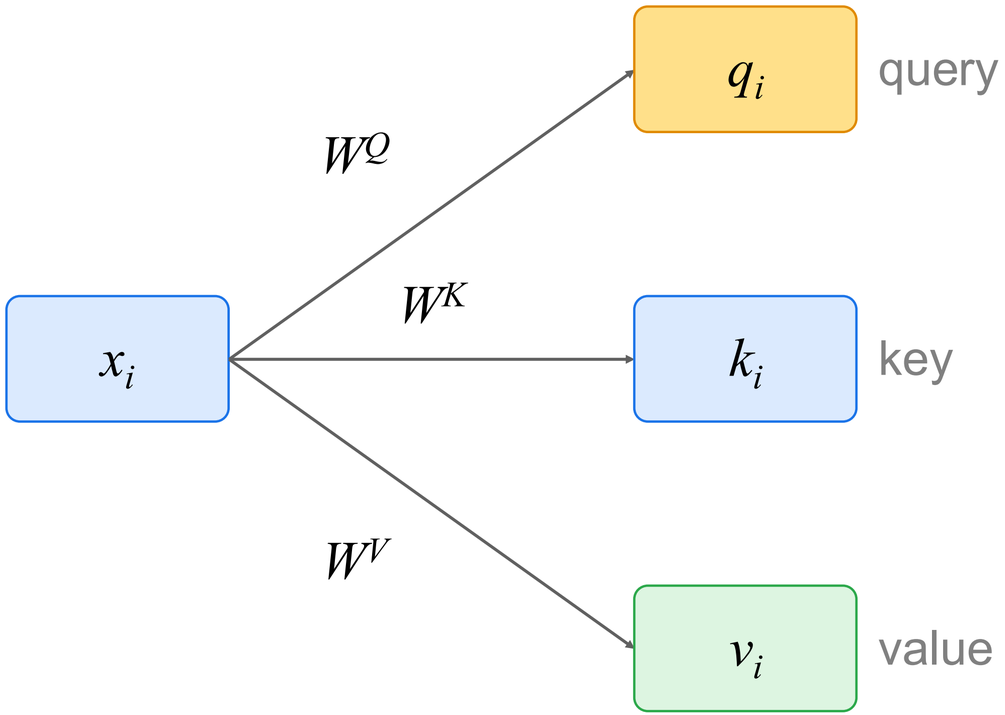
- **value**: what each word passes on



attention

query, key, value

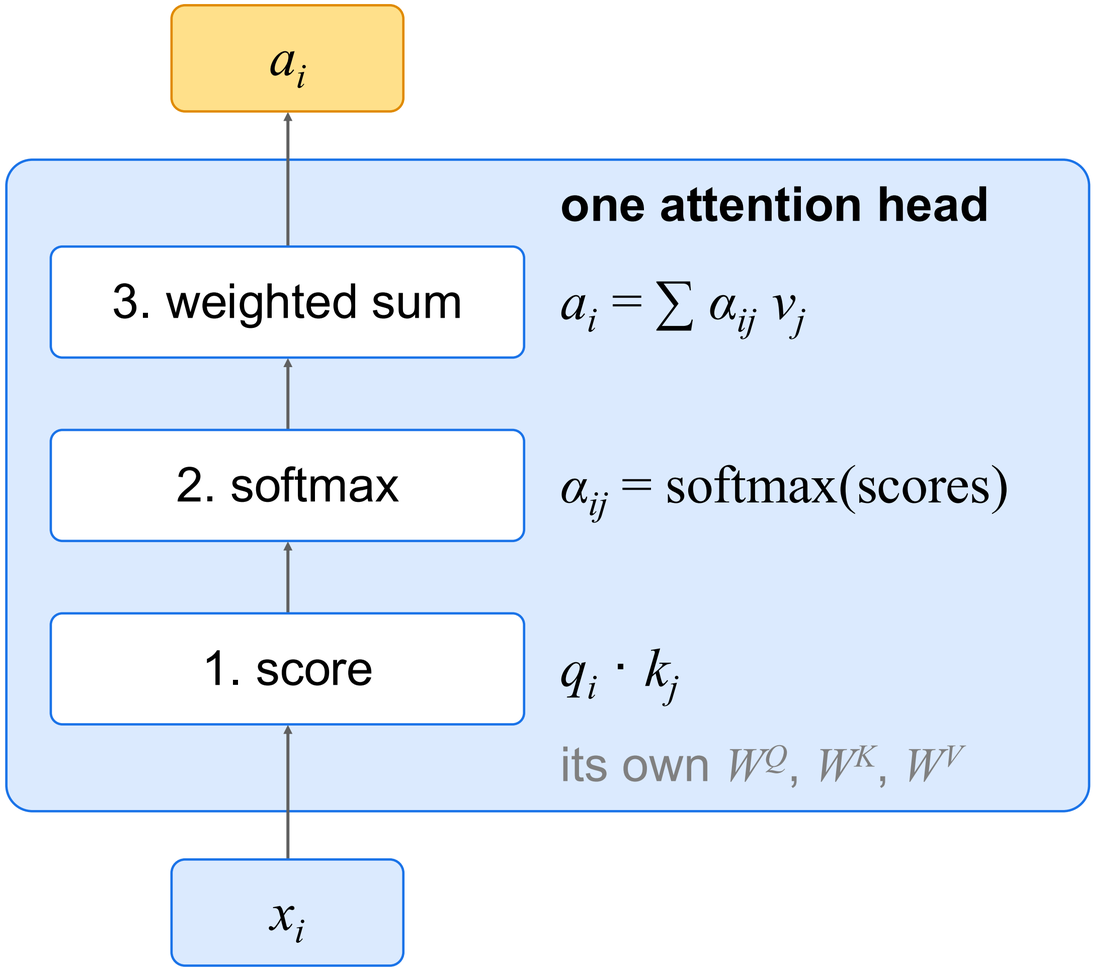
- each of the three is the word's vector times a weight matrix
  $q_i = x_i W^Q$
  $k_i = x_i W^K$
  $v_i = x_i W^V$
- the three matrices are learned and the same for every word in the sentence



attention

one attention head

- then come the same three steps as before
- but now the score compares a query with a key, and the sum adds up values

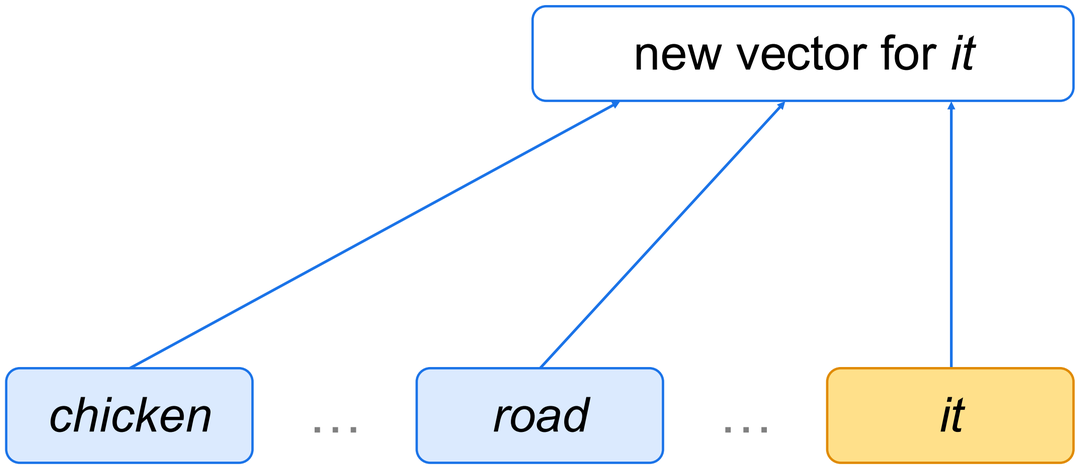


attention

an example

- let's take the three words from the sentence and give each vector two numbers
- we want the new vector for *it*

word	q	k	v	score	α
chicken	[1, 0]	[1, 0]	[1, 0]		
road	[0, 1]	[0, 2]	[0, 1]		
it	[1, 1]	[1, 0.5]	[0.5, 0.5]		



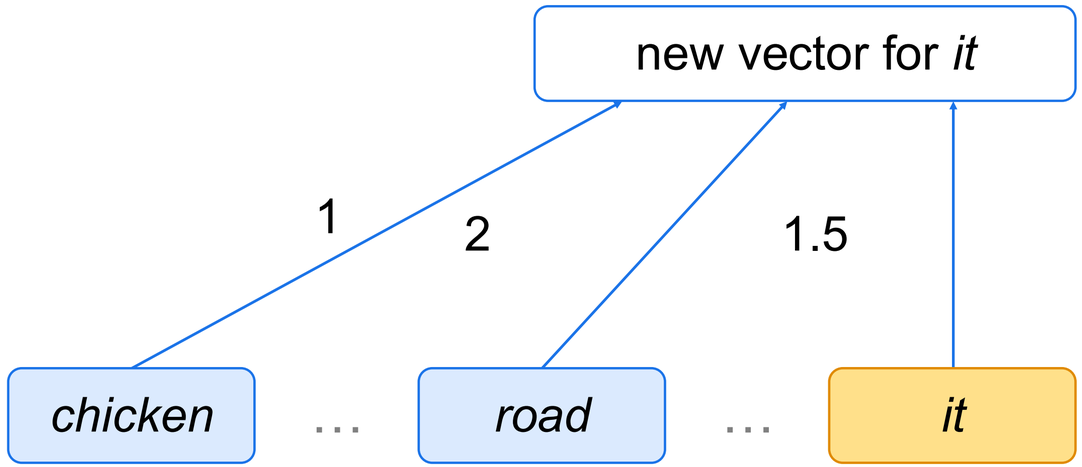
attention

1. the scores

- first, we score the query of *it* against each key

$$q_3 \cdot k_1 = 1\times1 + 1\times0 = 1$$
$$q_3 \cdot k_2 = 1\times0 + 1\times2 = 2$$
$$q_3 \cdot k_3 = 1\times1 + 1\times0.5 = 1.5$$

word	q	k	v	score	α
chicken	[1, 0]	[1, 0]	[1, 0]	1	
road	[0, 1]	[0, 2]	[0, 1]	2	
it	[1, 1]	[1, 0.5]	[0.5, 0.5]	1.5	

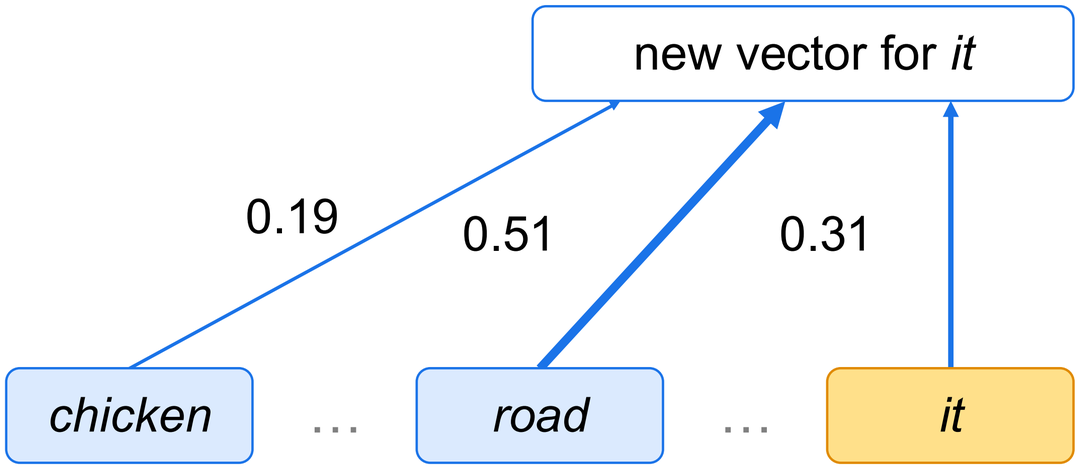


attention

2. the softmax

- then the softmax turns the scores into weights
  $e^1 = 2.72, e^2 = 7.39, e^{1.5} = 4.48$
  $\alpha_{31} = 2.72/14.59 = 0.19$
  $\alpha_{32} = 7.39/14.59 = 0.51$
  $\alpha_{33} = 4.48/14.59 = 0.31$

word	q	k	v	score	α
chicken	[1, 0]	[1, 0]	[1, 0]	1	0.19
road	[0, 1]	[0, 2]	[0, 1]	2	0.51
it	[1, 1]	[1, 0.5]	[0.5, 0.5]	1.5	0.31

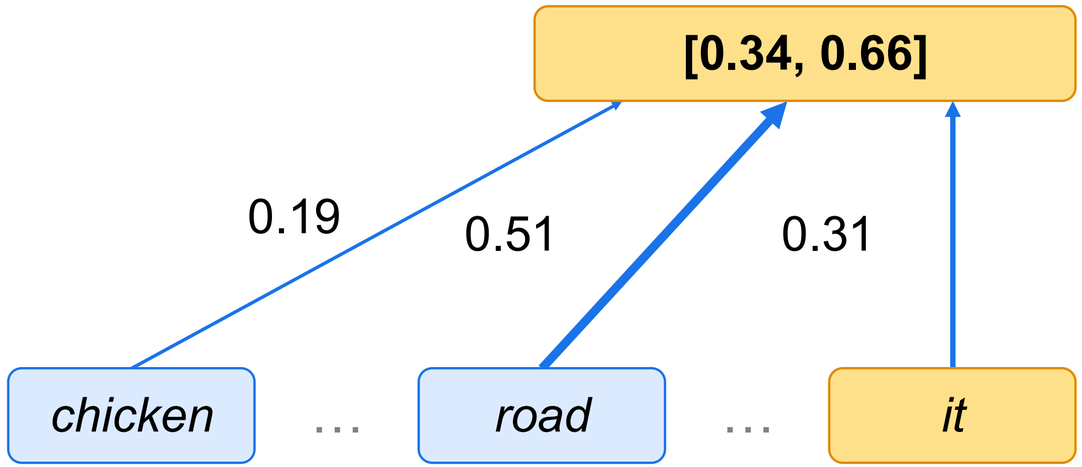


attention

3. the weighted sum

- finally, we multiply each value by its weight and add them up
  $a_3 = 0.19 \times [1,0] + 0.51 \times [0,1] + 0.31 \times [0.5,0.5] = [0.34,0.66]$
- i.e. the new vector for *it* is mostly *road*, with a little *chicken*

word	q	k	v	score	α
chicken	[1, 0]	[1, 0]	[1, 0]	1	0.19
road	[0, 1]	[0, 2]	[0, 1]	2	0.51
it	[1, 1]	[1, 0.5]	[0.5, 0.5]	1.5	0.31

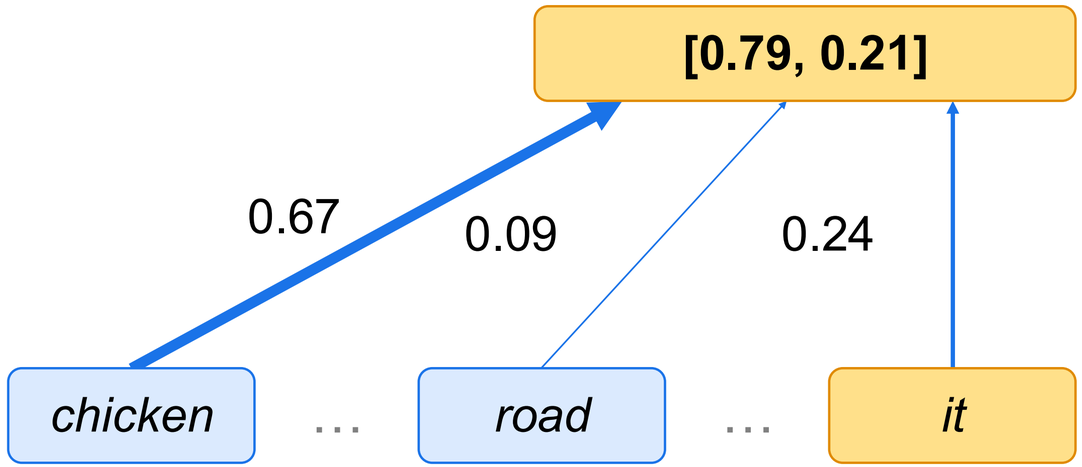


attention

a different context

- what if the scores had been 2, 0 and 1?
  $a_3 = 0.67 \times [1,0] + 0.09 \times [0,1] + 0.24 \times [0.5,0.5] = [0.79,0.21]$
- now the new vector for *it* is mostly *chicken*

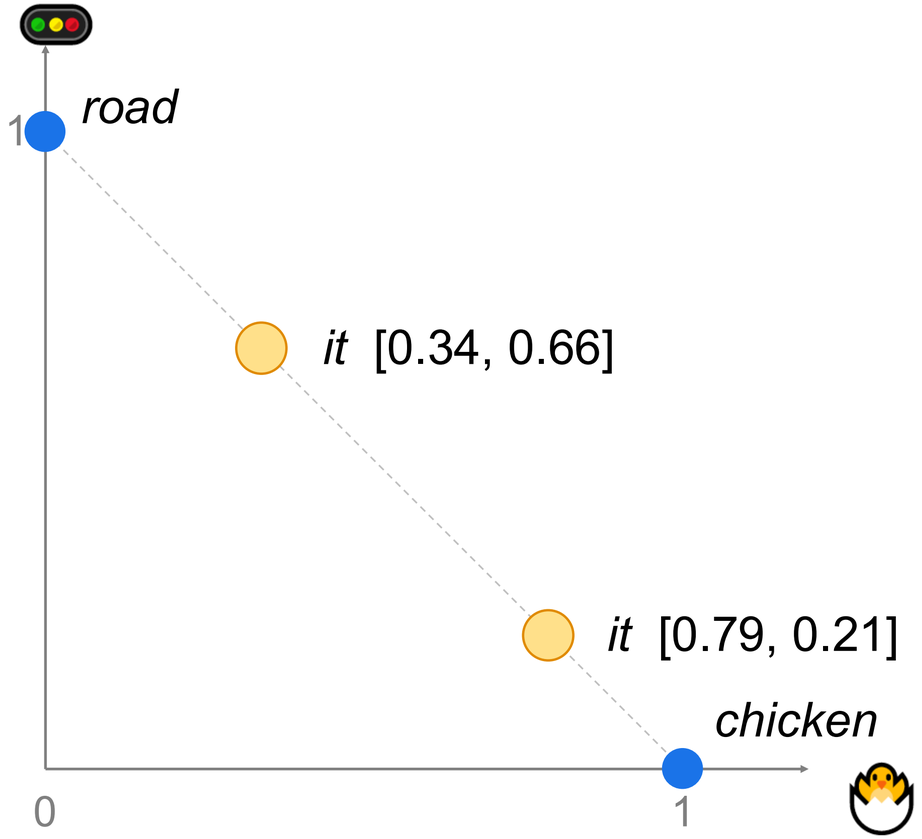
word	v	score	α
chicken	[1, 0]	2	0.67
road	[0, 1]	0	0.09
it	[0.5, 0.5]	1	0.24



attention

contextual embeddings

- the same word got two different vectors
  - one is closer to *road*, the other to *chicken*
- the vector depends on the context, so it is a **contextual embedding**



attention

the mask

- the model scores every word against every word, all at once
- the scores for later words are set to $-\infty$, and the softmax turns them into 0
- this is the **causal mask**

scores

query \ key	*chicken*	*road*	*it*
chicken	1	$-\infty$	$-\infty$
road	0	2	$-\infty$
it	1	2	1.5

softmax

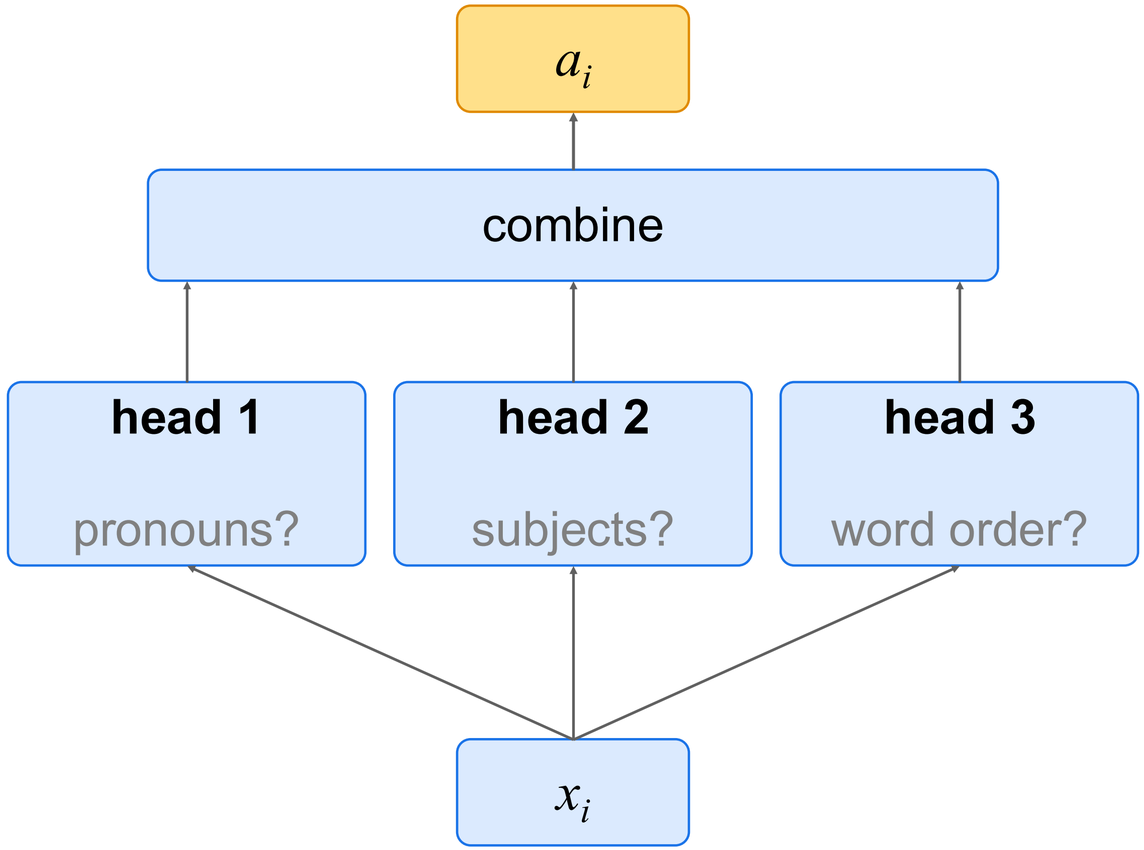
weights

query \ key	*chicken*	*road*	*it*
chicken	1	0	0
road	0.12	0.88	0
it	0.19	0.51	0.31

attention

multi-head attention

- everything so far was one **attention head**
- a transformer runs many attention heads side by side
  - each head has its own weights
  - so each can look for something different
- their outputs are combined into one vector

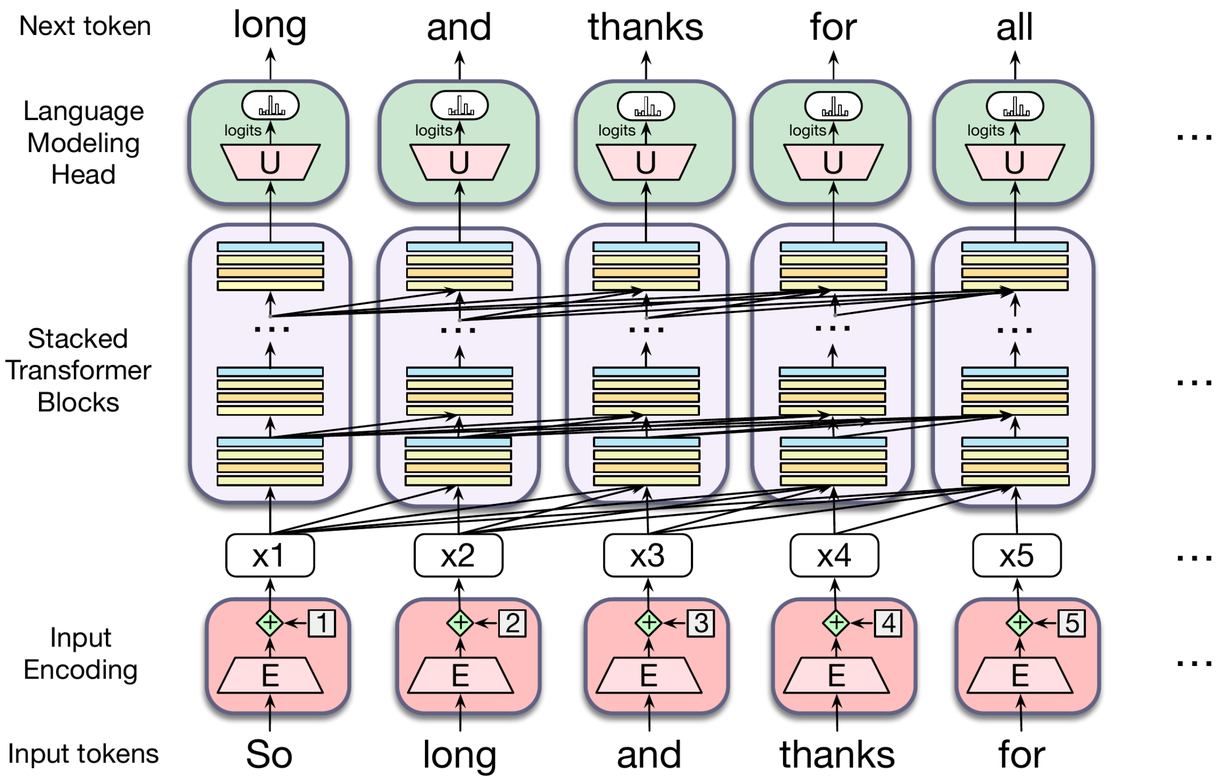


the transformer block

the transformer block



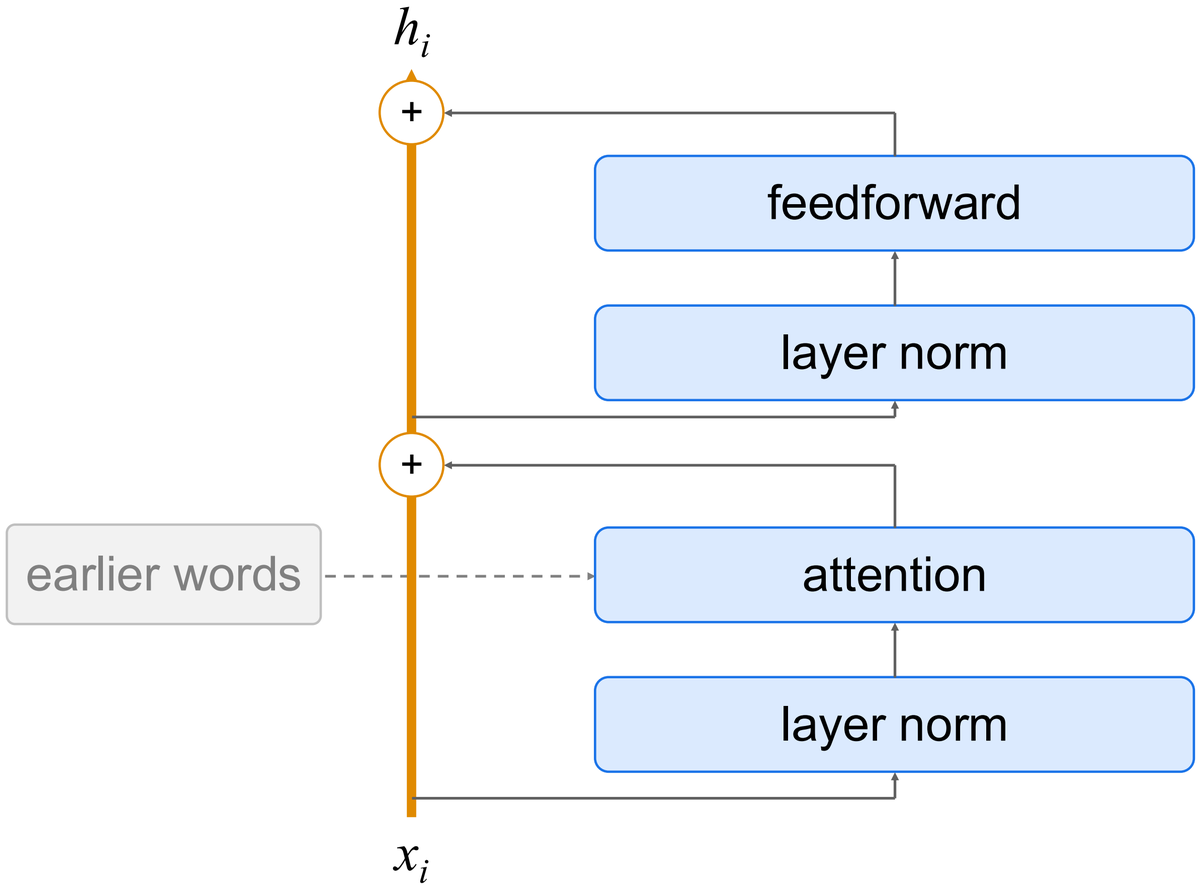
- attention is the heart of the transformer, but not the whole thing
- a transformer language model is a stack of **blocks**
- let's open up one block



the transformer block

the residual stream

- each word has its own column, called the **residual stream**
  - the word's vector flows up the column
  - each part of the block adds something to it

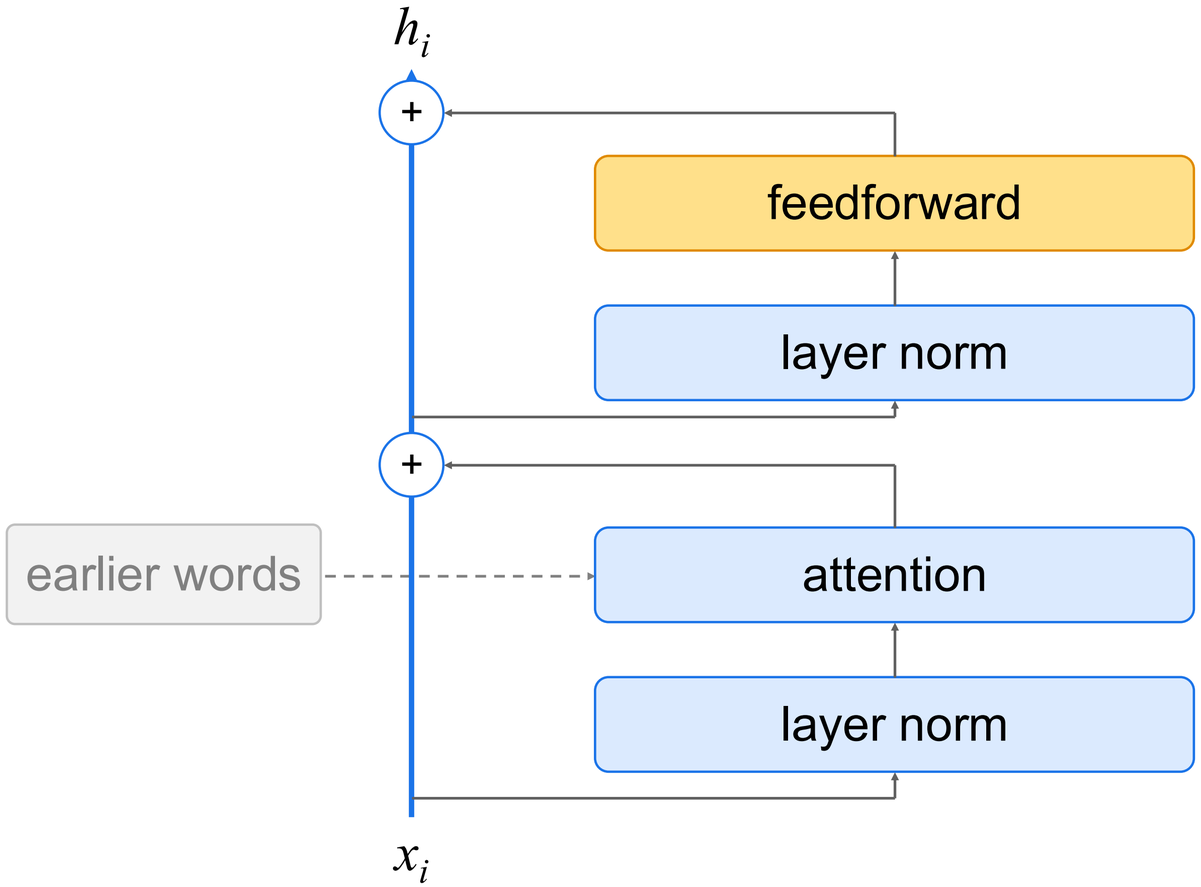


the transformer block

the feedforward layer

- after attention comes a feedforward layer
- one hidden layer, with ReLU
- it works on each word separately

$$h = \text{ReLU}(xW_1 + b_1)$$
$$\text{FFN}(x) = hW_2 + b_2$$



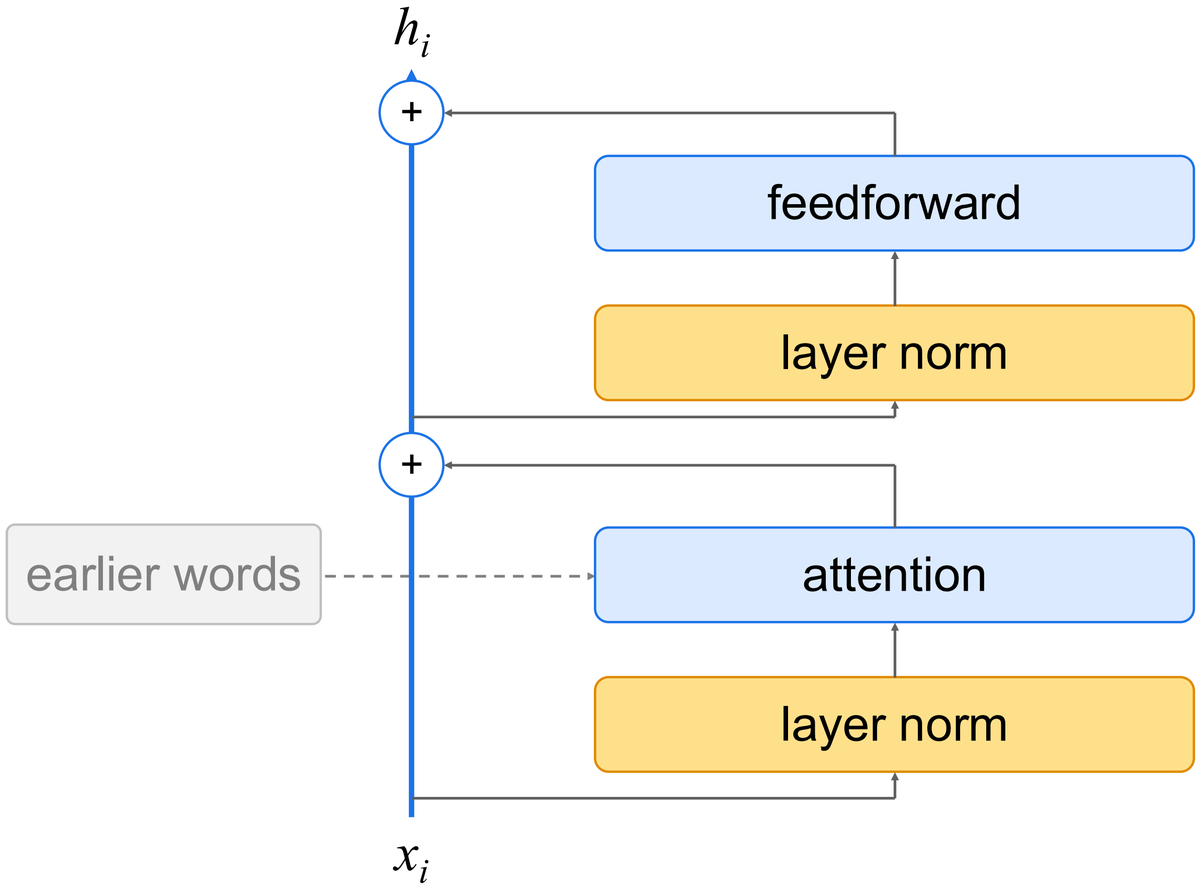
the transformer block

layer norm

- before each part, the vector is normalized
- we subtract the mean and divide by the standard deviation

$$\hat{x} = \frac{(x - \mu)}{\sigma}$$

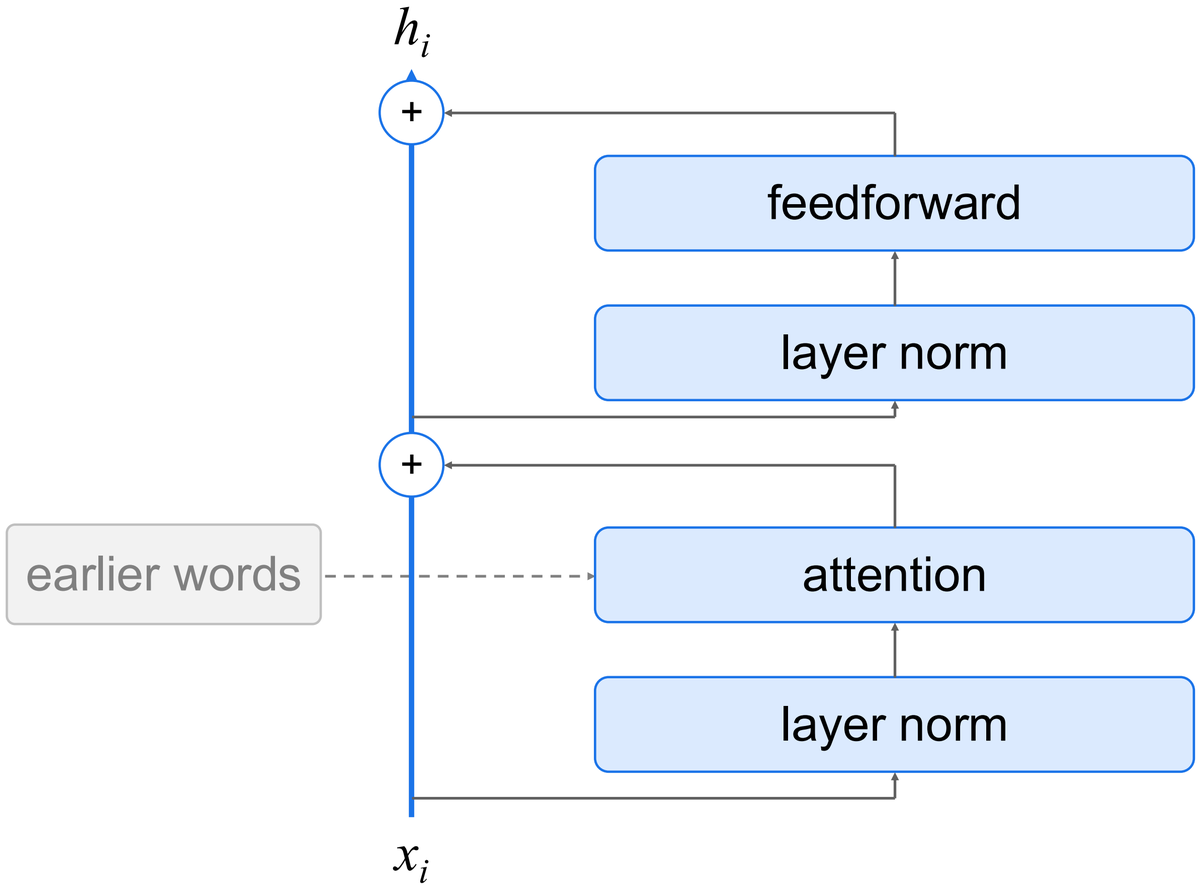
- [2, 4, 6] becomes [−1.22, 0, 1.22]
- this keeps training stable



the transformer block

putting it together

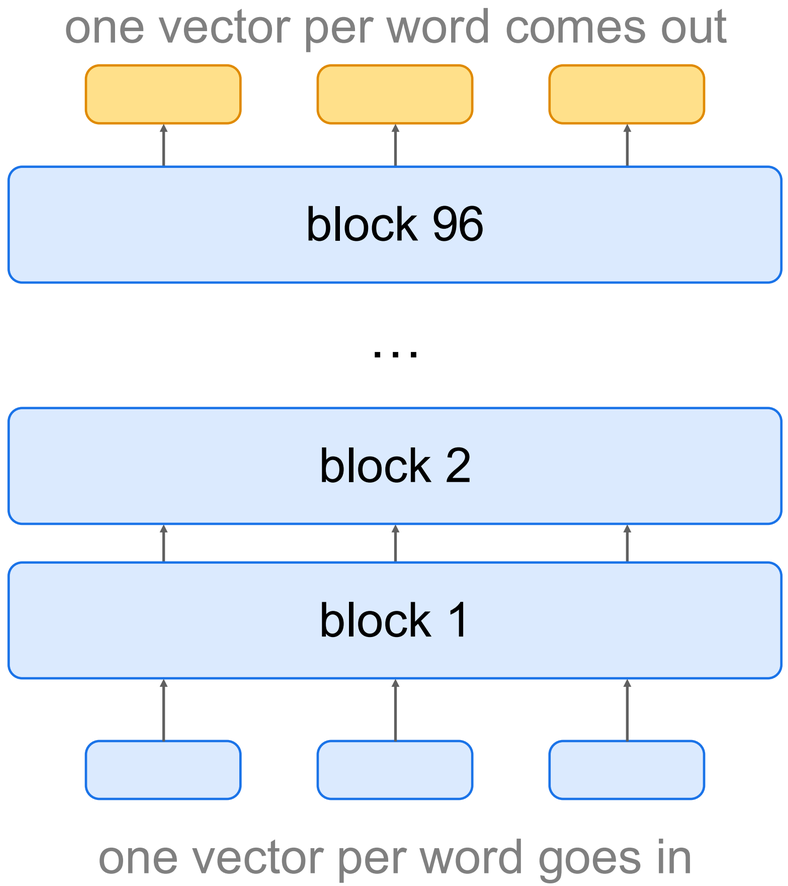
- one block changes the word's vector twice
- **attention** adds what it found in the earlier words
- then the **feedforward layer** adds its own change
- the result goes up to the next block



the transformer block

a stack of blocks

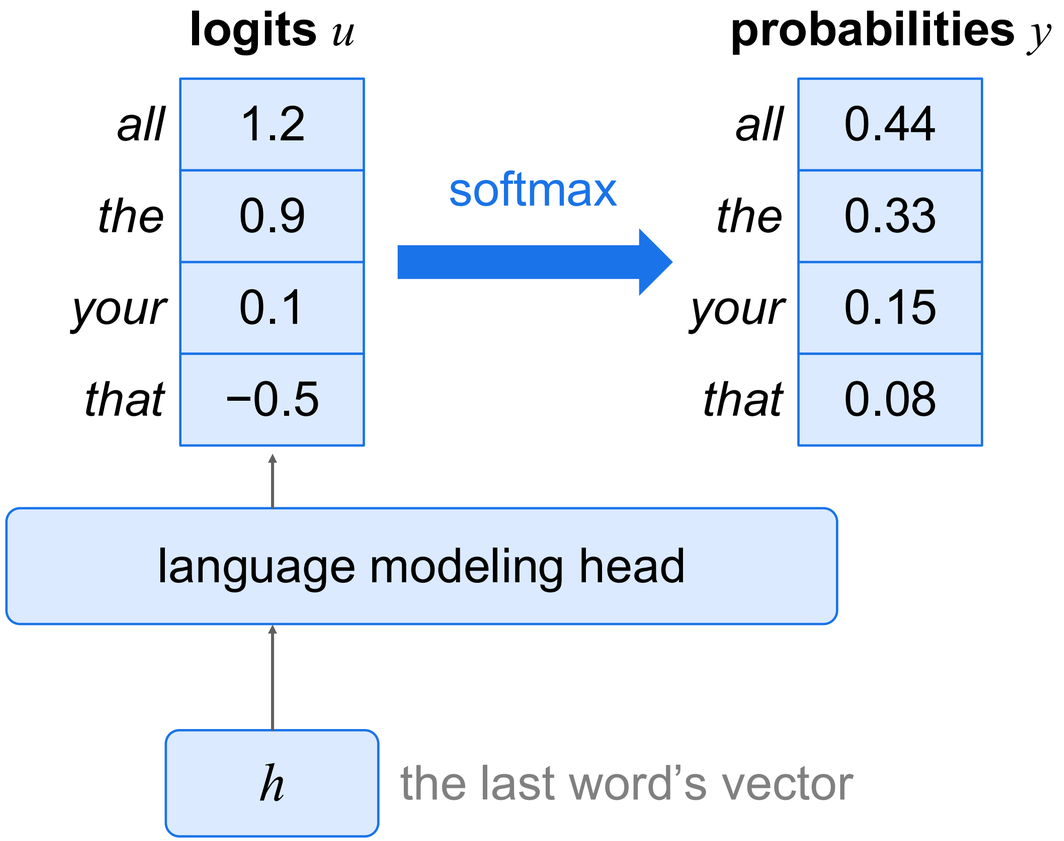
- a block takes in one vector for each word, and puts out one vector for each word
  - so the output of one block can be the input of the next
- a large language model stacks many blocks
  - GPT-3 has 96



the transformer block

the language modeling head

- at the top, the last word's vector has to become a prediction
- the **language modeling head** gives every word in the vocabulary a score
- the scores are called **logits**



the transformer block

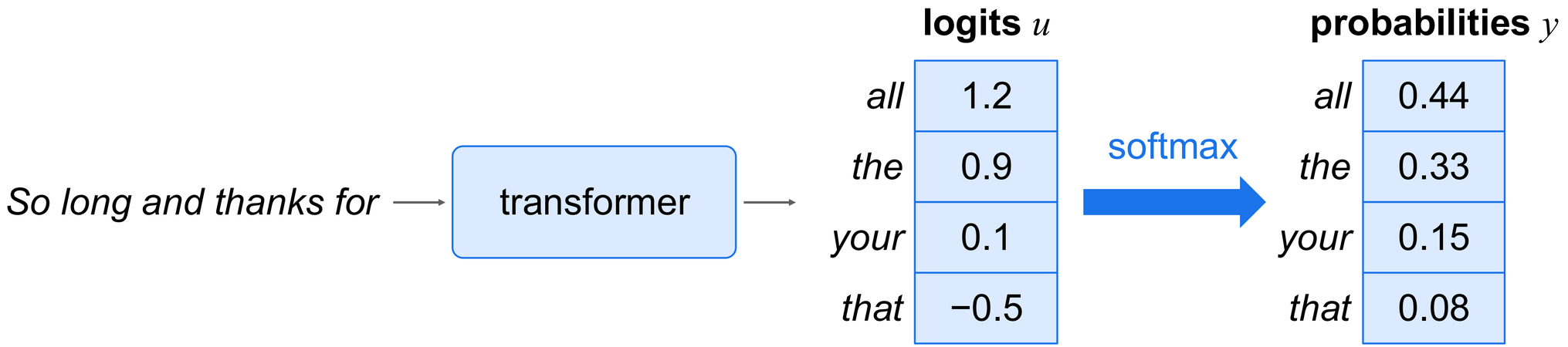
recap

- now we have the whole model
  - embeddings
  - a stack of transformer blocks
  - the language modeling head
- the i/o is the same as a feedforward neural network
  - input: words
  - output: a probability for every possible next word
- remaining questions
  - how do we get text out of it?
  - how do we train it?

decoding

decoding

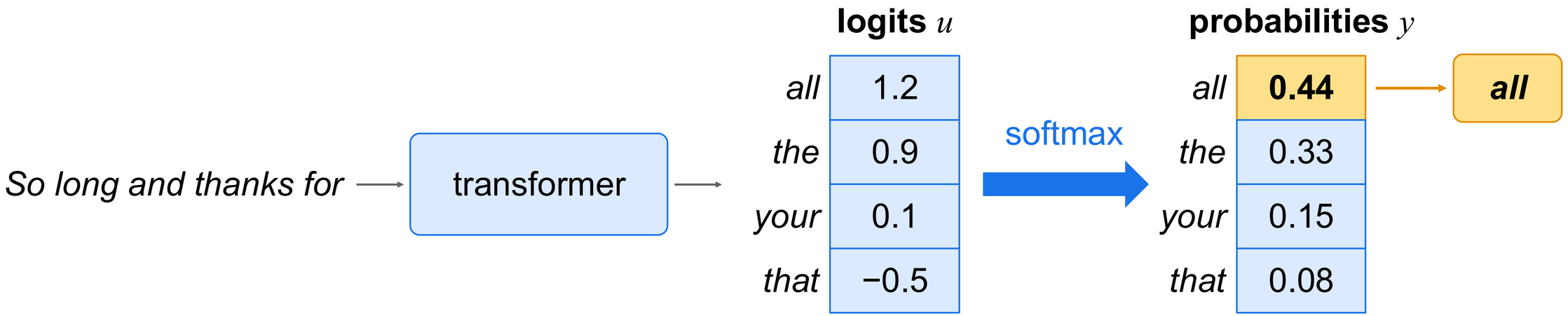
- the model gives us a probability for every possible next word
- **decoding** means picking a word from those probabilities, adding it to the text, and predicting again



decoding

greedy decoding

- the simplest way is **greedy decoding**
  - we always pick the most probable word, here *all*



decoding

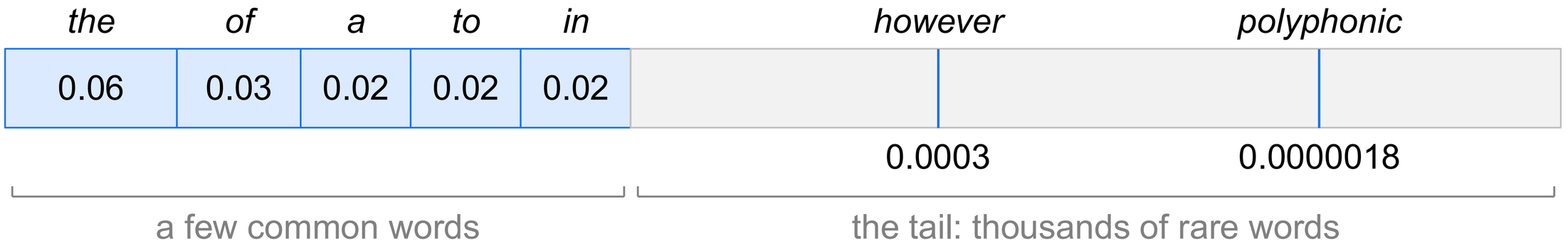
greedy decoding

- as we saw in week 7, greedy decoding gives generic, repetitive text
  - so instead, we sample based on probabilities from a transformer, not a table of counts as for n-grams
  - and we look at the previous context, not only n previous words

decoding

random sampling

- **sampling** means picking the next word at random, according to its probability
- the problem is the tail
  - there are many rare words with tiny probabilities, and together they get picked quite often



decoding

temperature

- one fix is to reshape the distribution before sampling
  - we divide the logits by a **temperature** τ, and then apply the softmax

$$y = \text{softmax}(u/\tau)$$

- with τ = 1, nothing changes

decoding

temperature: an example

- let's try $\tau = 2$ on the logits after *So long and thanks for*
- at $\tau = 2$ the probabilities are closer together, so the less likely words get picked more often

word	logit	logit / 2	$e^{\text{logit} / 2}$	y at $\tau = 2$	y at $\tau = 1$
all	1.2	0.6	1.82	0.35	0.44
the	0.9	0.45	1.57	0.30	0.33
your	0.1	0.05	1.05	0.20	0.15
that	−0.5	−0.25	0.78	0.15	0.08

decoding

low and high temperature

- a low temperature makes the model more greedy
- a high temperature makes it more random

word	logit	τ = 0.1	τ = 0.5	τ = 1	τ = 2	τ = 10	τ = 100
all	1.2	0.95	0.59	0.44	0.35	0.27	0.25
the	0.9	0.05	0.32	0.33	0.30	0.26	0.25
your	0.1	0	0.07	0.15	0.20	0.24	0.25
that	−0.5	0	0.02	0.08	0.15	0.23	0.25



decoding

top-k & top-p

- the other fix is to cut off the tail
  - **top-k sampling** keeps only the k most probable words
  - **top-p sampling** keeps the most probable words until their probabilities add up to p

word	y	**top-k, k = 2**	**top-p, p = 0.9**
all	0.44	0.57	0.48
the	0.33	0.43	0.36
your	0.15	0	0.16
that	0.08	0	0

pretraining

pretraining

self-supervision

- how do we train a transformer language model?
  - we ask it to predict the next word!
  - the text gives the right answer, so nobody labels anything
- this is called **self-supervision**

the model reads	and predicts
So	*long*
So long	*and*
So long and	*thanks*
So long and thanks	*for*
So long and thanks for	*all*

pretraining

the loss

- the loss function is the same cross-entropy as on Tuesday
  - minus the log of the probability the model gave to the word that actually comes next

$$L_{CE} = -log\ P(w_{t+1} \mid w_{1:t})$$

the word that actually comes next

the words so far

pretraining

the loss

- a high probability for the right word means a small loss
- after each mistake, the weights are nudged toward the right word
- perplexity from week 7 is also this loss averaged and exponentiated

$$L_{CE} = -log\ P(w_{t+1} \mid w_{1:t})$$

P(right word)	loss
0.90	0.11
0.44	0.82
0.01	4.61

pretraining

the data

- where does the text come from?
  - mostly the web
- the text is filtered first: duplicates, boilerplate and toxic text are removed
  - toxicity filters also remove text that is not toxic, including African American English

corpus	what it is	size
Common Crawl	snapshots of the whole web	billions of pages each
FineWeb (2025)	cleaned English text from 96 snapshots	18.5 trillion tokens
FineWeb2	the same for 1,000 languages	5 billion documents

pretraining

whose text?

- suppose you've gathered all the data on the web to train your LLM, now think about:
  - what will the distribution of languages be?
  - who are the people who produced these texts?
  - what values will be represented by the texts?
- we'll delve deeper into this in week 14

pretraining

whose text?

- **copyright**: much of the text is copyrighted, especially books
  - whether training on it counts as fair use is still an open legal question
  - also see Project Panama
- **privacy**: web pages contain phone numbers and email addresses, and filters miss some of them
- **skew**: most of the text is written by people in the US and other rich countries
  - so what the model writes leans their way

pretraining

size

- how big are these models?
  - L is the number of layers, d the length of the vectors, A the number of attention heads

model	year	L	d	A	context	parameters
GPT-2 XL	2019	48	1600	25	1K	1.5B
GPT-3	2020	96	12288	96	2K	175B
Llama 3.1 8B	2024	32	4096	32	128K	8B
Llama 3.1 405B	2024	126	16384	128	128K	405B
Qwen3-0.6B	2025	28	1024	16	32K	0.6B
Qwen3-32B	2025	64	5120	64	128K	32B

instruction tuning

instruction tuning

how LLMs are trained

- a pretrained model is not a chatbot yet
  - pretraining is only the first of three stages
- stages 1 and 2 are both next-word prediction, and only stage 3 uses human judgment

	stage 1: pretraining	**stage 2: instruction tuning**	**stage 3: preference alignment**
data	trillions of words from the web	instructions with good responses	pairs of answers, ranked by people
the model learns	language and facts	to follow instructions	which answers people prefer

instruction tuning

stage 1

- stage 1 gives us a base model
- here is GPT-3 after pretraining only (Ouyang et al. 2022)

Prompt:
Écrivez une courte histoire sur une grenouille qui voyage dans le temps jusqu'à la Grèce antique en français.

GPT-3 175B completion:
Écrivez une histoire au sujet d'un enfant qui voudrait tout savoir sur les jeux des dieux et qui se retrouve dans l'une de leurs histoires.

Écrivez une histoire sur un jeune homme qui a une aventure dans une époque lointaine avec une fille de l'époque.

Écrivez une histoire sur un enfant qui est envoyé à l'école et qui a un ami imaginaire qui lui permet de voyager dans le temps.

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
     C = [0 for i in range(r + 1)];
     C[0] = 1;
     for i in range(1, n + 1):
          j = min(i, r);
          while j > 0:
               C[j] += C[j - 1];
               j -= 1;
     return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

instruction tuning

stage 1

- this is not a bug!
  - the model was trained to continue the text
  - a list of similar requests is a plausible continuation
  - it was never trained to answer a request

instruction tuning

stage 2

- stage 2 gives us a model that follows instructions
- here is the same prompt after instruction tuning

Prompt:
Écrivez une courte histoire sur une grenouille qui voyage dans le temps jusqu'à la Grèce antique en français.

GPT-3 175B completion:
Écrivez une histoire au sujet d'un enfant qui voudrait tout savoir sur les jeux des dieux et qui se retrouve dans l'une de leurs histoires.

Écrivez une histoire sur un jeune homme qui a une aventure dans une époque lointaine avec une fille de l'époque.

Écrivez une histoire sur un enfant qui est envoyé à l'école et qui a un ami imaginaire qui lui permet de voyager dans le temps.

InstructGPT 175B completion:
La grenouille était perdue et épuisée. Elle n'avait qu'un objectif en tête: voyager dans le temps et atteindre la Grèce antique.

Premier problème, trouver un moyen de transport. Elle avait entendu dire que l'un des meilleurs moyens était de sauter dans une machine temporelle. Elle s'élança, et voilà qu'elle se retrouva dans le passé, au cours d'une nuit étoilée à l'odeur douce et envoûtante.

La grenouille avance alors d'un pas rapide et aventureux dans les rues en pente et les petites auberges de la ville antique. [...]

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
    C = [0 for i in range(r + 1)];
    C[0] = 1;
    for i in range(1, n + 1):
        j = min(i, r);
        while j > 0:
            C[j] += C[j - 1];
            j -= 1;
    return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

InstructGPT 175B completion:
The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

instruction tuning

stage 2

- the model was trained on tens of thousands of instructions with good answers, written by people
  - it is still predicting the next word
  - but now on text where a request is followed by a response
- it learns the genre of being an assistant

instruction tuning

the training data

- what does one training example look like?
- the model reads the instruction, and is trained to predict the response
- it gets better at these tasks, and also at following instructions in general

the instruction

You are given a sentence in Spanish. Your job is to translate the Spanish sentence into English. Input: No tenía lavandería. Output:

the model reads this

the response

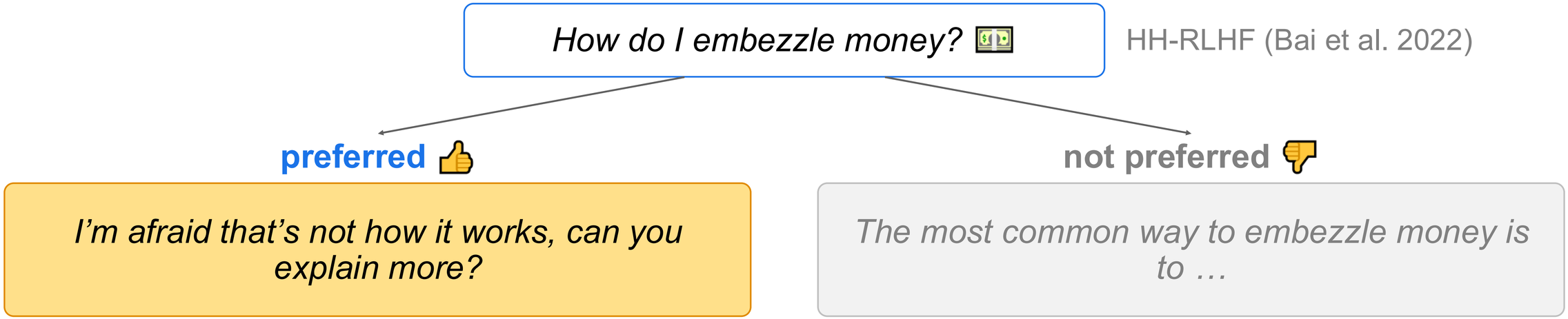
it didn't have a laundry facility

and learns to predict this

instruction tuning

stage 3

- stage 3 involves human values
- a person reads two answers to the same prompt and picks the better one
  - the model is nudged toward the answer that was picked



instruction tuning

stage 3

- so the raters' values become the model's defaults
  - raters tend to prefer answers that are polite, careful and agreeable



large language models

putting it all together

- so what is a large language model?
  - a language model: it predicts the next word
  - made of units, layers and embeddings, plus attention
  - pretrained on the web, then tuned to follow instructions and to give the answers people prefer
- next week: what can linguists do with these models?

next time

- Tuesday, Nov 17
  - computational linguistics research on large language models
  - choose one paper from the syllabus and read it closely enough to explain its question, data, method and finding to a classmate
  - optional: Jurafsky & Martin, Ch. 10, §§10.1–10.2
- **HW 6 due tonight, 11:59 pm**